

Coverage Metric - GPU Optimized

Karina Arichavala

2026-05-11

Table of contents

0.1	1. Setup e Imports	1
0.2	2. Cargar Datos	2
0.3	3. Cargar SAE y Extraer Features	3
0.4	4. Filtrar BSPs de Piezas	4
0.5	5. Implementación de Coverage	5
0.5.1	Estrategia de optimización:	5
0.6	6. Calcular Coverage	8
0.7	7. Análisis de Resultados	8
0.8	8. Guardar Resultados	35
0.9	9. Generar PDF con Quarto	35

```
# Configuración para visualización en PDF
import pandas as pd
import numpy as np
import sys

# Para que el texto de pandas no se corte
pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 100) # Ancho fijo para pandas

# Para prints largos de numpy - ancho más conservador
np.set_printoptions(linewidth=100, edgeitems=3)

# Configurar ancho de terminal para prints
import os
os.environ['COLUMNS'] = '100'

print(" Configuración para PDF lista")
```

Configuración para PDF lista

0.1 1. Setup e Imports

```
import numpy as np
import torch
import sys
import os
from pathlib import Path
from tqdm import tqdm
import time

# Configurar paths
project_root = Path('../..').resolve()
sys.path.insert(0, str(project_root))
```

```

from sae.models.sae import SparseAutoencoder
from sae.tools.naming_utils import get_checkpoint_dir, get_model_name, get_extras_id
from sae.tools.experiment_utils import get_experiment_dir, get_metrics_dir, get_report_name

# Verificar GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memoria disponible: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

```

Device: cuda
 GPU: NVIDIA GeForce RTX 5060
 Memoria disponible: 8.55 GB

```

# Metadata para naming de checkpoints (separada de hiperparámetros)
NAMING_META = {
    'variante': 'standard',          # Arquitectura del SAE
    'tecnica': 'l1-decay',          # L1 con penalty annealing
    'capa': 6,                      # Layer del modelo OthelloGPT
    'juegos': 1500,                 # Número de partidas
    'base_path': 'C:\\Users\\Esposa\\Documents\\Repos\\sae-othello-gpt', # Path base para checkpoints
    'experiment_base_path': os.path.abspath('.././experiments'), # sae/experiments/
    'extras': {
        'expansion': 32,
        'sparsity_coef': 1e-2,
        'epochs': 1000,
        'patience': 20,
        'learning_rate': 1e-3,
        'sparsity_decay': 0.98,
        'batch_size': 256
    }
}
print('\n Metadata del experimento:')
for key, value in NAMING_META.items():
    print(f' {key}: {value}')

```

Metadata del experimento:

```

variante: standard
tecnica: l1-decay
capa: 6
juegos: 1500
base_path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt
experiment_base_path: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments
extras: {'expansion': 32, 'sparsity_coef': 0.01, 'epochs': 1000, 'patience': 20, 'learning_rate': 0.001, 'sparsity_de

```

0.2 2. Cargar Datos

```

# Cargar activaciones pre-SAE
activations_path = project_root / "sae" / "activations" / "data" / "layer6_200games.npy"
activations_full = np.load(activations_path)
####
activations = activations_full

print(f"Activaciones del modelo:")
print(f" Shape original: {activations_full.shape}")
print(f" Shape final: {activations.shape}")
print(f" Memoria: {activations.nbytes / (1024**2):.2f} MB")

```

Activaciones del modelo:

Shape original: (11800, 512)

Shape final: (11800, 512)

Memoria: 23.05 MB

```
# Cargar ground truth de BSPs
bsp_gt_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_200games.npy"
bsp_names_path = project_root / "sae" / "metrics" / "02_data" / "bsp_ground_truth_200games.names.npy"

bsp_ground_truth_full = np.load(bsp_gt_path)
bsp_names = np.load(bsp_names_path, allow_pickle=True)
###
indices = np.random.choice(len(activations_full), size=1000, replace=False)
activations = activations_full[indices]
##
bsp_ground_truth = bsp_ground_truth_full[indices]
print(f"\nGround truth BSPs:")
print(f" Shape original: {bsp_ground_truth_full.shape}")
print(f" Shape final: {bsp_ground_truth.shape}")
print(f" Total BSPs: {len(bsp_names)}")
```

Ground truth BSPs:

Shape original: (11800, 326)

Shape final: (1000, 326)

Total BSPs: 326

0.3 3. Cargar SAE y Extraer Features

```
# Configuración del SAE
input_dim = 512
hidden_dim = 16384

# Construir ruta del modelo usando naming_utils
checkpoint_dir = get_checkpoint_dir(
    NAMING_META['base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos']
)
extras_id = get_extras_id(checkpoint_dir, NAMING_META['extras']) if NAMING_META.get('extras') else None

model_name = get_model_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    'best',
    extras_id=extras_id
)
model_path = checkpoint_dir / model_name

# Cargar modelo (weights_only=False: checkpoint contiene objetos Path en config)
sae = SparseAutoencoder(input_dim, hidden_dim).to(device)
checkpoint = torch.load(model_path, map_location=device, weights_only=False)
sae.load_state_dict(checkpoint['model_state_dict'])
sae.eval()

print(f"SAE cargado:")
```

```

print(f" Path: {model_path}")
print(f" Input: {input_dim}, Hidden: {hidden_dim}")
print(f" Expansion: {hidden_dim/input_dim}x")
print(f" Epoch: {checkpoint.get('epoch', 'N/A')}, Val MSE: {checkpoint.get('val_mse', float('nan')):.6f}")

```

SAE cargado:

```

Path: C:\Users\Esposa\Documents\Repos\sae-othello-gpt\layer_06\models\sae_standard\sae_l1-decay_standard\1500games\sae_standard\
decay_16_1500g_ex1_best.pt
Input: 512, Hidden: 16384
Expansion: 32.0x
Epoch: N/A, Val MSE: nan

```

```

print("Extrayendo features del SAE...")
activations_tensor = torch.from_numpy(activations).float().to(device)

# PRIMERO: calcular sae_features con los 15 tableros
with torch.no_grad():
    sae_features = torch.relu(sae.encoder(activations_tensor))

# SEGUNDO: calcular f_max_global sobre TODO el dataset
activations_full_tensor = torch.from_numpy(activations_full).float().to(device)
with torch.no_grad():
    sae_features_full = torch.relu(sae.encoder(activations_full_tensor))
f_max_global = sae_features_full.max(dim=0)[0]

# Liberar memoria
del activations_full_tensor, sae_features_full
torch.cuda.empty_cache()
print("f_max global calculado sobre todo el dataset")

print(f"\nFeatures SAE:")
print(f" Shape: {sae_features.shape}")
print(f" Device: {sae_features.device}")
print(f" Sparsity: {(sae_features == 0).float().mean():.2%}")
print(f" Activaciones promedio: {(sae_features > 0).sum(dim=1).float().mean():.1f}")

```

Extrayendo features del SAE...

f_max global calculado sobre todo el dataset

Features SAE:

```

Shape: torch.Size([1000, 16384])
Device: cuda:0
Sparsity: 99.84%
Activaciones promedio: 26.4

```

0.4 4. Filtrar BSPs de Piezas

Coverage solo usa las 128 BSPs de piezas (mías/oponente), sin vacías.

```

# =====
# CONFIGURACIÓN DE FILTRADO DE BSPs
# =====
USE_PLAYER   = False # BSPs perspectiva del jugador (1 y 2)
USE_ABSOLUTE = True  # BSPs absolutas negro/blanco (B y W)
USE_STRATEGIC = False # BSPs especiales (BSP_)

# Filtrar según configuración
bsp_piezas_indices = []
bsp_piezas_names = []

```

```

for i, name in enumerate(bsp_names):
    include = False

    if USE_PLAYER and len(name) == 6 and name.startswith('BSP') and (name.endswith('1') or name.endswith('2')):
        include = True
    if USE_ABSOLUTE and (name.endswith('B') or name.endswith('W')):
        include = True
    if USE_STRATEGIC and name.startswith('BSP_'):
        include = True

    if include:
        bsp_pieces_indices.append(i)
        bsp_pieces_names.append(name)

bsp_pieces_indices = np.array(bsp_pieces_indices)
bsp_pieces_gt = bsp_ground_truth[:, bsp_pieces_indices]

# Convertir a tensor en GPU
bsp_pieces_gt_tensor = torch.from_numpy(bsp_pieces_gt).bool().to(device)

print(f"BSPs seleccionadas para Coverage:")
print(f" Player: {USE_PLAYER} | Absolute: {USE_ABSOLUTE} | Strategic: {USE_STRATEGIC}")
print(f" Total: {len(bsp_pieces_indices)}")
print(f" Shape: {bsp_pieces_gt_tensor.shape}")
print(f" Device: {bsp_pieces_gt_tensor.device}")
print(f" Primeras 5: {'', '.join(bsp_pieces_names[:5])}")
print(f" Últimas 5: {'', '.join(bsp_pieces_names[-5:])}")
# Identificador del filtrado (para naming de archivos de salida)
parts = []
if USE_PLAYER: parts.append("player")
if USE_ABSOLUTE: parts.append("absolute")
if USE_STRATEGIC: parts.append("strategic")
filter_suffix = "_".join(parts) if parts else "none"
print(f" Filter suffix: {filter_suffix}")

```

```

BSPs seleccionadas para Coverage:
Player: False | Absolute: True | Strategic: False
Total: 128
Shape: torch.Size([1000, 128])
Device: cuda:0
Primeras 5: BSPA1B, BSPA1W, BSPA2B, BSPA2W, BSPA3B
Últimas 5: BSPH6W, BSPH7B, BSPH7W, BSPH8B, BSPH8W
Filter suffix: absolute

```

0.5 5. Implementación de Coverage

0.5.1 Estrategia de optimización:

1. **Vectorización:** Procesar múltiples features en paralelo usando operaciones matriciales
2. **Batch processing:** Dividir en chunks para no saturar memoria GPU
3. **Pre-cálculo:** Calcular máximos una sola vez
4. **Early stopping:** Terminar cuando F1 = 1.0

```

def fast_f1_score_gpu(y_true, y_pred, eps=1e-8):
    """
    F1 score vectorizado en GPU.

    Args:
        y_true: Tensor (n_positions,) booleano
        y_pred: Tensor (n_positions, n_candidates) booleano
    """

```

```

Returns:
    f1_scores: Tensor (n_candidates,)
"""
# y_true: (n_positions,) -> expand to (n_positions, 1)
y_true_expanded = y_true.unsqueeze(1) # (n_positions, 1)

# Calcular TP, FP, FN
tp = (y_true_expanded & y_pred).sum(dim=0).float() # (n_candidates,)
fp = (~y_true_expanded & y_pred).sum(dim=0).float()
fn = (y_true_expanded & ~y_pred).sum(dim=0).float()

# Precision y Recall
precision = tp / (tp + fp + eps)
recall = tp / (tp + fn + eps)

# F1
f1 = 2 * (precision * recall) / (precision + recall + eps)

return f1

def calculate_coverage_gpu_optimized(
    sae_features, # Tensor (n_positions, n_features) en GPU
    bsp_ground_truth, # Tensor (n_positions, n_bsps) en GPU, bool
    thresholds=[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9],
    batch_size_features=512, # Procesar 512 features a la vez
    f_max=None, # ← AGREGAR
    verbose=True
):
    """
    Calcula Coverage de forma optimizada para GPU.

    Procesa múltiples features y thresholds en paralelo usando operaciones matriciales.
    """
    n_positions, n_features = sae_features.shape
    n_bsps = bsp_ground_truth.shape[1]
    n_thresholds = len(thresholds)

    if verbose:
        print("="*60)
        print("CALCULATING COVERAGE (GPU OPTIMIZED)")
        print("="*60)
        print(f"Positions: {n_positions:,}")
        print(f"SAE Features: {n_features:,}")
        print(f"BSPs: {n_bsps:,}")
        print(f"Thresholds: {n_thresholds:,}")
        print(f"Batch size (features): {batch_size_features:,}")
        print(f"Total evaluations: {n_bsps * n_features * n_thresholds:,}")
        print("="*60)

    # Pre-calcular máximos de features
    if f_max is None:
        f_max = sae_features.max(dim=0)[0]
    active_features = (f_max > 0).nonzero(as_tuple=True)[0]
    n_active = len(active_features)

    if verbose:
        print(f"\nActive features: {n_active:,}/{n_features:,} ({n_active/n_features*100:.1f}%)")
        print(f"Reduced evaluations: {n_bsps * n_active * n_thresholds:,}")
        print()

    # Almacenar resultados

```

```

best_f1s = torch.zeros(n_bsps, device=sae_features.device)
best_features = torch.full((n_bsps,), -1, dtype=torch.long, device=sae_features.device)
best_thresholds = torch.full((n_bsps,), -1.0, device=sae_features.device)

thresholds_tensor = torch.tensor(thresholds, device=sae_features.device)

# Procesar cada BSP
pbar = tqdm(range(n_bsps), desc="Processing BSPs") if verbose else range(n_bsps)

for bsp_idx in pbar:
    bsp_labels = bsp_ground_truth[:, bsp_idx] # (n_positions,)

    # Skip si la BSP nunca está activa
    if not bsp_labels.any():
        continue

    # Procesar features activas en batches
    for batch_start in range(0, n_active, batch_size_features):
        batch_end = min(batch_start + batch_size_features, n_active)
        batch_features_idx = active_features[batch_start:batch_end]

        # Extraer features del batch
        batch_activations = sae_features[:, batch_features_idx] # (n_positions, batch_size)
        batch_f_max = f_max[batch_features_idx] # (batch_size,)

        # Para cada threshold, binarizar TODAS las features del batch a la vez
        for t in thresholds_tensor:
            # Binarizar: (n_positions, batch_size)
            predictions = batch_activations > (t * batch_f_max.unsqueeze(0))

            # Calcular F1 para todas las features del batch
            f1_scores = fast_f1_score_gpu(bsp_labels, predictions) # (batch_size,)

            # Encontrar la mejor en este batch
            max_f1, max_idx_in_batch = f1_scores.max(dim=0)

            # Actualizar si es mejor que lo visto hasta ahora
            if max_f1 > best_f1s[bsp_idx]:
                best_f1s[bsp_idx] = max_f1
                best_features[bsp_idx] = batch_features_idx[max_idx_in_batch]
                best_thresholds[bsp_idx] = t

        # Early stopping: si ya es casi perfecto, no revisar más batches
        if best_f1s[bsp_idx] >= 0.999:
            break

# Calcular Coverage (macro-average)
coverage = best_f1s.mean().item()

# Convertir a CPU para retornar
best_f1s_cpu = best_f1s.cpu().numpy()
best_features_cpu = best_features.cpu().numpy()
best_thresholds_cpu = best_thresholds.cpu().numpy()

return coverage, best_f1s_cpu, best_features_cpu, best_thresholds_cpu

print(" Funciones de Coverage GPU definidas")

```

Funciones de Coverage GPU definidas

0.6 6. Calcular Coverage

```
# Medir tiempo de ejecución
start_time = time.time()

coverage, best_f1s, best_features, best_thresholds = calculate_coverage_gpu_optimized(
    sae_features,
    bsp_pieces_gt_tensor,
    batch_size_features=512, # Ajustar según memoria GPU
    f_max=f_max_global,
    verbose=True
)

elapsed_time = time.time() - start_time

print("\n" + "="*60)
print("RESULTADO COVERAGE")
print("="*60)
print(f"Coverage Score: {coverage:.4f}")
print(f"Objetivo (paper): 0.52")
print(f"Diferencia: {coverage - 0.52:+.4f}")
print(f"\nTiempo de ejecución: {elapsed_time:.2f} seg")
print(f"({elapsed_time/60:.2f} min)")
print("="*60)
```

```
=====
CALCULATING COVERAGE (GPU OPTIMIZED)
=====
```

```
Positions: 1,000
SAE Features: 16,384
BSPs: 128
Thresholds: 10
Batch size (features): 512
Total evaluations: 20,971,520
=====
```

```
Active features: 13,463/16,384 (82.2%)
Reduced evaluations: 17,232,640
```

```
Processing BSPs: 0% | 0/128 [00:00<?, ?it/s] Processing BSPs: 100% | 128/128 [00:23<00:00, 5.50it/s]
```

```
=====
RESULTADO COVERAGE
=====
```

```
Coverage Score: 0.4678
Objetivo (paper): 0.52
Diferencia: -0.0522
```

```
Tiempo de ejecución: 23.30 seg
(0.39 min)
=====
```

0.7 7. Análisis de Resultados


```
# Estadísticas de distribución de F1 scores
print("Distribución de F1 scores:")
print(f" Mínimo: {best_f1s.min():.4f}")
print(f" Máximo: {best_f1s.max():.4f}")
print(f" Media: {best_f1s.mean():.4f}")
print(f" Mediana: {np.median(best_f1s):.4f}")
print(f" Std: {best_f1s.std():.4f}")
print()
print(f"BSPs con F1 > 0.8: {(best_f1s > 0.8).sum()} ({(best_f1s > 0.8).mean()*100:.1f}%)")
print(f"BSPs con F1 > 0.5: {(best_f1s > 0.5).sum()} ({(best_f1s > 0.5).mean()*100:.1f}%)")
print(f"BSPs con F1 < 0.2: {(best_f1s < 0.2).sum()} ({(best_f1s < 0.2).mean()*100:.1f}%)")
```

Distribución de F1 scores:

```
Mínimo: 0.3128
Máximo: 0.7235
Media: 0.4678
Mediana: 0.4474
Std: 0.0858
```

```
BSPs con F1 > 0.8: 0 (0.0%)
BSPs con F1 > 0.5: 33 (25.8%)
BSPs con F1 < 0.2: 0 (0.0%)
```

```
# Top 10 BSPs mejor detectadas
top_indices = np.argsort(best_f1s)[-10:][::-1]

print("\nTop 10 BSPs mejor detectadas:")
print("="*60)
print(f"{'BSP':<10} {'F1':<8} {'Feature':<10} {'Threshold':<10}")
print("-"*60)
for idx in top_indices:
    bsp_name = bsp_pieces_names[idx]
    f1 = best_f1s[idx]
    feat = best_features[idx]
    thresh = best_thresholds[idx]
    print(f"{bsp_name:<10} {f1:<8.4f} {feat:<10} {thresh:<10.1f}")
```

Top 10 BSPs mejor detectadas:

```
=====
```

BSP	F1	Feature	Threshold
BSPE4B	0.7235	13051	0.0
BSPD4B	0.6983	14514	0.0
BSPD5W	0.6773	4049	0.0
BSPE5B	0.6731	15238	0.0
BSPE5W	0.6658	14514	0.0
BSPD5B	0.6594	13629	0.0
BSPD4W	0.6332	15238	0.0
BSPD3B	0.6317	14400	0.0
BSPC5W	0.6139	11238	0.0
BSPF4W	0.6102	1225	0.0

```
# Bottom 10 BSPs peor detectadas
bottom_indices = np.argsort(best_f1s)[:10]

print("\nBottom 10 BSPs peor detectadas:")
print("="*60)
print(f"{'BSP':<10} {'F1':<8} {'Feature':<10} {'Threshold':<10}")
print("-"*60)
for idx in bottom_indices:
```

```

bsp_name = bsp_pieces_names[idx]
f1 = best_fis[idx]
feat = best_features[idx]
thresh = best_thresholds[idx]
print(f"{bsp_name:<10} {f1:<8.4f} {feat:<10} {thresh:<10.1f}")

```

Bottom 10 BSPs peor detectadas:

```

=====
BSP          F1          Feature    Threshold
-----
BSPB8B       0.3128       4049         0.2
BSPA8B       0.3133       4049         0.3
BSPA7B       0.3385       4049         0.2
BSPE1W       0.3421       4049         0.1
BSPH5B       0.3431       4049         0.2
BSPH7B       0.3506       4049         0.2
BSPD1W       0.3571       2121         0.0
BSPH4B       0.3606       4049         0.2
BSPB7B       0.3650       2182         0.1
BSPA2W       0.3660       4049         0.2

```

```

# Distribución de thresholds óptimos
unique_thresholds, counts = np.unique(best_thresholds, return_counts=True)

print("\nDistribución de thresholds óptimos:")
print("="*60)
for t, count in zip(unique_thresholds, counts):
    if t >= 0: # Ignorar -1 (BSPs sin match)
        percentage = count / len(best_thresholds) * 100
        print(f"Threshold {t:.1f}: {count:3d} BSPs ({percentage:.1f}%)")

```

Distribución de thresholds óptimos:

```

=====
Threshold 0.0:  57 BSPs (44.5%)
Threshold 0.1:  16 BSPs (12.5%)
Threshold 0.2:  45 BSPs (35.2%)
Threshold 0.3:  10 BSPs (7.8%)

```

```

# =====
# CONSULTA POR BSP ESPECÍFICA
# =====

# >>> CAMBIA AQUÍ LA BSP QUE QUIERES ANALIZAR <
BSP_OBJETIVO = "BSPE4B"

# Buscar índice de la BSP
bsp_idx = bsp_pieces_names.index(BSP_OBJETIVO)

# Extraer resultados ya calculados
neurona  = best_features[bsp_idx]
threshold = best_thresholds[bsp_idx]
f1       = best_fis[bsp_idx]

# f_max de la neurona ganadora (sae_features ya está en memoria)
f_max_neurona = sae_features[:, neurona].max().item()

# Activaciones de esa neurona en todos los tableros
activaciones_neurona = sae_features[:, neurona].cpu().numpy()

```

```

print(f"BSP analizada:      {BSP_OBJETIVO}")
print(f"Neurona ganadora:   {neurona}")
print(f"Threshold óptimo:    {threshold:.1f}")
print(f"F_max neurona:       {f_max_neurona:.4f}")
print(f"Umbral efectivo:      {threshold:.1f} x {f_max_neurona:.4f} = {threshold * f_max_neurona:.4f}")
print(f"F1 score:           {f1:.4f}")
print(f"\nActivaciones de neurona {neurona} ({len(activaciones_neurona)} tableros):")
for i, act in enumerate(activaciones_neurona):
    print(f"  Tablero {i+1:5d}: {act:.6f}")

```

```

BSP analizada:      BSPE4B
Neurona ganadora:   13051
Threshold óptimo:    0.0
F_max neurona:       6.5474
Umbral efectivo:     0.0 x 6.5474 = 0.0000
F1 score:           0.7235

```

Activaciones de neurona 13051 (1000 tableros):

```

Tablero  1: 0.129396
Tablero  2: 0.000000
Tablero  3: 0.093055
Tablero  4: 0.198065
Tablero  5: 0.852410
Tablero  6: 0.044564
Tablero  7: 0.124082
Tablero  8: 0.136946
Tablero  9: 1.913908
Tablero 10: 0.298517
Tablero 11: 0.103117
Tablero 12: 0.174797
Tablero 13: 0.229159
Tablero 14: 0.153650
Tablero 15: 0.201598
Tablero 16: 0.522417
Tablero 17: 0.102879
Tablero 18: 1.202558
Tablero 19: 0.599440
Tablero 20: 0.440541
Tablero 21: 0.689938
Tablero 22: 0.213772
Tablero 23: 0.480972
Tablero 24: 0.198315
Tablero 25: 0.134253
Tablero 26: 2.047186
Tablero 27: 0.057620
Tablero 28: 0.055589
Tablero 29: 0.116226
Tablero 30: 0.131832
Tablero 31: 0.048492
Tablero 32: 0.256711
Tablero 33: 0.000000
Tablero 34: 0.156122
Tablero 35: 0.129658
Tablero 36: 0.154153
Tablero 37: 0.198485
Tablero 38: 0.055810
Tablero 39: 0.039637
Tablero 40: 0.157277
Tablero 41: 0.097857
Tablero 42: 0.072760
Tablero 43: 0.446564
Tablero 44: 0.199836

```

Tablero	45:	0.076250
Tablero	46:	0.544132
Tablero	47:	0.013916
Tablero	48:	0.147231
Tablero	49:	0.089844
Tablero	50:	0.082308
Tablero	51:	0.188361
Tablero	52:	0.151906
Tablero	53:	0.144969
Tablero	54:	0.111490
Tablero	55:	0.118233
Tablero	56:	0.141012
Tablero	57:	0.115839
Tablero	58:	0.152885
Tablero	59:	0.191668
Tablero	60:	0.083760
Tablero	61:	0.173360
Tablero	62:	0.190056
Tablero	63:	0.465632
Tablero	64:	0.005621
Tablero	65:	0.177120
Tablero	66:	0.209133
Tablero	67:	0.040985
Tablero	68:	0.386654
Tablero	69:	0.126570
Tablero	70:	0.103838
Tablero	71:	0.110185
Tablero	72:	0.130426
Tablero	73:	0.167259
Tablero	74:	0.131592
Tablero	75:	0.056137
Tablero	76:	0.090733
Tablero	77:	0.000000
Tablero	78:	0.117305
Tablero	79:	0.150310
Tablero	80:	0.440541
Tablero	81:	0.137605
Tablero	82:	0.111376
Tablero	83:	0.145974
Tablero	84:	0.160201
Tablero	85:	0.137940
Tablero	86:	0.053379
Tablero	87:	0.274132
Tablero	88:	0.109263
Tablero	89:	0.174201
Tablero	90:	0.102555
Tablero	91:	0.083196
Tablero	92:	0.102709
Tablero	93:	0.247824
Tablero	94:	0.069679
Tablero	95:	0.140220
Tablero	96:	0.126687
Tablero	97:	0.183832
Tablero	98:	0.168879
Tablero	99:	0.068541
Tablero	100:	0.291505
Tablero	101:	0.512263
Tablero	102:	0.111445
Tablero	103:	0.064819
Tablero	104:	0.132678
Tablero	105:	0.135294
Tablero	106:	0.070208

Tablero	107:	0.221822
Tablero	108:	0.071158
Tablero	109:	0.222689
Tablero	110:	0.040624
Tablero	111:	0.033756
Tablero	112:	0.086282
Tablero	113:	1.491965
Tablero	114:	0.127030
Tablero	115:	0.139574
Tablero	116:	0.145885
Tablero	117:	1.202558
Tablero	118:	0.170329
Tablero	119:	0.210717
Tablero	120:	0.196400
Tablero	121:	0.205702
Tablero	122:	0.828269
Tablero	123:	0.440541
Tablero	124:	0.127761
Tablero	125:	0.000000
Tablero	126:	0.115509
Tablero	127:	0.090149
Tablero	128:	0.140934
Tablero	129:	0.015267
Tablero	130:	0.286278
Tablero	131:	0.144323
Tablero	132:	0.208071
Tablero	133:	0.215577
Tablero	134:	0.089067
Tablero	135:	0.002589
Tablero	136:	2.047186
Tablero	137:	0.121996
Tablero	138:	0.168051
Tablero	139:	0.023053
Tablero	140:	0.220904
Tablero	141:	0.079672
Tablero	142:	0.034875
Tablero	143:	0.566418
Tablero	144:	0.462926
Tablero	145:	0.409506
Tablero	146:	0.186346
Tablero	147:	0.203595
Tablero	148:	0.091123
Tablero	149:	0.118379
Tablero	150:	0.413935
Tablero	151:	0.297014
Tablero	152:	0.083905
Tablero	153:	0.075652
Tablero	154:	0.122529
Tablero	155:	0.167302
Tablero	156:	0.038929
Tablero	157:	0.084244
Tablero	158:	0.083194
Tablero	159:	0.505287
Tablero	160:	0.139925
Tablero	161:	0.215302
Tablero	162:	0.259138
Tablero	163:	0.061392
Tablero	164:	0.250139
Tablero	165:	0.051324
Tablero	166:	0.351851
Tablero	167:	0.290629
Tablero	168:	0.019241

Tablero	169:	0.106977
Tablero	170:	0.109971
Tablero	171:	0.000000
Tablero	172:	0.100223
Tablero	173:	0.463847
Tablero	174:	0.453769
Tablero	175:	0.127614
Tablero	176:	0.107501
Tablero	177:	0.294813
Tablero	178:	0.000000
Tablero	179:	0.008629
Tablero	180:	0.112600
Tablero	181:	0.343681
Tablero	182:	0.021771
Tablero	183:	0.352025
Tablero	184:	0.110444
Tablero	185:	0.146383
Tablero	186:	0.046474
Tablero	187:	0.109757
Tablero	188:	0.354307
Tablero	189:	0.098902
Tablero	190:	0.179318
Tablero	191:	0.131581
Tablero	192:	0.077787
Tablero	193:	0.078279
Tablero	194:	0.106302
Tablero	195:	0.077474
Tablero	196:	0.200575
Tablero	197:	0.109934
Tablero	198:	0.135438
Tablero	199:	0.099253
Tablero	200:	0.174466
Tablero	201:	0.480476
Tablero	202:	0.276750
Tablero	203:	0.224728
Tablero	204:	0.099688
Tablero	205:	0.204821
Tablero	206:	0.161563
Tablero	207:	6.547399
Tablero	208:	1.202558
Tablero	209:	0.000000
Tablero	210:	0.128613
Tablero	211:	0.070808
Tablero	212:	0.000000
Tablero	213:	0.149924
Tablero	214:	6.547399
Tablero	215:	0.183118
Tablero	216:	0.003678
Tablero	217:	0.169082
Tablero	218:	0.226457
Tablero	219:	0.118049
Tablero	220:	0.136670
Tablero	221:	0.023391
Tablero	222:	0.016838
Tablero	223:	0.026454
Tablero	224:	0.106322
Tablero	225:	0.141559
Tablero	226:	0.114551
Tablero	227:	0.153677
Tablero	228:	0.200604
Tablero	229:	0.154782
Tablero	230:	0.244005

Tablero	231:	0.453246
Tablero	232:	0.151867
Tablero	233:	0.238816
Tablero	234:	0.148811
Tablero	235:	0.041637
Tablero	236:	0.040302
Tablero	237:	0.175868
Tablero	238:	0.075153
Tablero	239:	0.111739
Tablero	240:	0.049389
Tablero	241:	0.163670
Tablero	242:	0.143871
Tablero	243:	0.070419
Tablero	244:	0.019251
Tablero	245:	0.195696
Tablero	246:	0.273029
Tablero	247:	1.163766
Tablero	248:	0.473872
Tablero	249:	0.126796
Tablero	250:	0.046892
Tablero	251:	0.306697
Tablero	252:	0.104302
Tablero	253:	0.115444
Tablero	254:	0.131753
Tablero	255:	0.243428
Tablero	256:	0.606705
Tablero	257:	0.142460
Tablero	258:	0.375025
Tablero	259:	0.088425
Tablero	260:	0.000000
Tablero	261:	0.064569
Tablero	262:	0.827311
Tablero	263:	0.165955
Tablero	264:	0.169715
Tablero	265:	0.265291
Tablero	266:	0.147741
Tablero	267:	0.434833
Tablero	268:	0.465632
Tablero	269:	0.073447
Tablero	270:	0.073637
Tablero	271:	0.092978
Tablero	272:	0.164492
Tablero	273:	0.141715
Tablero	274:	0.203210
Tablero	275:	0.195572
Tablero	276:	0.000000
Tablero	277:	0.035936
Tablero	278:	0.113809
Tablero	279:	0.218977
Tablero	280:	0.107540
Tablero	281:	0.219763
Tablero	282:	0.191107
Tablero	283:	0.144772
Tablero	284:	0.327460
Tablero	285:	0.170688
Tablero	286:	0.203625
Tablero	287:	0.172668
Tablero	288:	0.042850
Tablero	289:	0.144391
Tablero	290:	0.135771
Tablero	291:	0.023247
Tablero	292:	0.101038

Tablero	293:	0.459976
Tablero	294:	0.329922
Tablero	295:	0.341998
Tablero	296:	0.288381
Tablero	297:	0.067056
Tablero	298:	0.113638
Tablero	299:	0.083651
Tablero	300:	0.600774
Tablero	301:	0.131487
Tablero	302:	0.465632
Tablero	303:	0.278922
Tablero	304:	0.027006
Tablero	305:	0.176948
Tablero	306:	0.156952
Tablero	307:	0.295857
Tablero	308:	0.166358
Tablero	309:	0.440541
Tablero	310:	0.197542
Tablero	311:	0.085346
Tablero	312:	0.101557
Tablero	313:	0.450846
Tablero	314:	0.184689
Tablero	315:	0.038715
Tablero	316:	0.081769
Tablero	317:	1.163766
Tablero	318:	0.154933
Tablero	319:	0.141340
Tablero	320:	0.167920
Tablero	321:	0.043754
Tablero	322:	0.244534
Tablero	323:	0.330772
Tablero	324:	0.024092
Tablero	325:	0.049809
Tablero	326:	0.133159
Tablero	327:	0.363136
Tablero	328:	0.040281
Tablero	329:	0.252456
Tablero	330:	0.076956
Tablero	331:	0.324055
Tablero	332:	0.140421
Tablero	333:	0.103612
Tablero	334:	0.060484
Tablero	335:	0.296149
Tablero	336:	0.113348
Tablero	337:	0.295482
Tablero	338:	0.138618
Tablero	339:	0.060735
Tablero	340:	0.092909
Tablero	341:	0.542256
Tablero	342:	0.100900
Tablero	343:	0.103189
Tablero	344:	0.123368
Tablero	345:	0.208011
Tablero	346:	0.703696
Tablero	347:	0.160695
Tablero	348:	0.093205
Tablero	349:	0.090852
Tablero	350:	0.059072
Tablero	351:	0.080679
Tablero	352:	0.185627
Tablero	353:	0.079417
Tablero	354:	0.049091

Tablero	355:	0.008534
Tablero	356:	0.050532
Tablero	357:	0.069689
Tablero	358:	0.329700
Tablero	359:	0.467735
Tablero	360:	0.075592
Tablero	361:	0.242561
Tablero	362:	0.110682
Tablero	363:	0.190789
Tablero	364:	0.124315
Tablero	365:	0.106715
Tablero	366:	0.151899
Tablero	367:	0.103034
Tablero	368:	0.000000
Tablero	369:	0.166764
Tablero	370:	0.073706
Tablero	371:	0.074300
Tablero	372:	0.176997
Tablero	373:	0.196410
Tablero	374:	0.156875
Tablero	375:	0.191600
Tablero	376:	0.168326
Tablero	377:	0.127959
Tablero	378:	0.074701
Tablero	379:	0.087676
Tablero	380:	0.049772
Tablero	381:	0.176541
Tablero	382:	0.243101
Tablero	383:	0.054474
Tablero	384:	0.061363
Tablero	385:	0.157215
Tablero	386:	0.060279
Tablero	387:	0.129417
Tablero	388:	0.204763
Tablero	389:	0.138896
Tablero	390:	0.040066
Tablero	391:	0.116166
Tablero	392:	0.212786
Tablero	393:	0.074607
Tablero	394:	0.174279
Tablero	395:	0.190531
Tablero	396:	0.058318
Tablero	397:	0.165162
Tablero	398:	0.025236
Tablero	399:	0.222826
Tablero	400:	0.093222
Tablero	401:	0.184203
Tablero	402:	0.154452
Tablero	403:	0.053589
Tablero	404:	0.039049
Tablero	405:	0.411109
Tablero	406:	0.326312
Tablero	407:	0.043612
Tablero	408:	0.902349
Tablero	409:	0.083307
Tablero	410:	0.440541
Tablero	411:	2.047186
Tablero	412:	0.025431
Tablero	413:	0.166309
Tablero	414:	0.129421
Tablero	415:	0.122491
Tablero	416:	0.087121

Tablero	417:	0.054283
Tablero	418:	0.090757
Tablero	419:	0.000000
Tablero	420:	0.175324
Tablero	421:	0.233078
Tablero	422:	0.208246
Tablero	423:	0.244757
Tablero	424:	0.081529
Tablero	425:	0.379236
Tablero	426:	0.070726
Tablero	427:	0.117131
Tablero	428:	0.002675
Tablero	429:	0.152635
Tablero	430:	0.004264
Tablero	431:	0.465632
Tablero	432:	0.248001
Tablero	433:	0.047288
Tablero	434:	0.064377
Tablero	435:	0.486028
Tablero	436:	0.066976
Tablero	437:	0.165649
Tablero	438:	0.000000
Tablero	439:	0.109287
Tablero	440:	0.020730
Tablero	441:	0.228510
Tablero	442:	0.592106
Tablero	443:	0.030976
Tablero	444:	0.120534
Tablero	445:	0.133217
Tablero	446:	0.158610
Tablero	447:	0.111625
Tablero	448:	0.515887
Tablero	449:	0.119142
Tablero	450:	0.223501
Tablero	451:	0.221821
Tablero	452:	0.262223
Tablero	453:	0.031323
Tablero	454:	0.147642
Tablero	455:	0.083150
Tablero	456:	0.362239
Tablero	457:	0.077318
Tablero	458:	0.066954
Tablero	459:	0.047078
Tablero	460:	0.150547
Tablero	461:	0.000000
Tablero	462:	0.000000
Tablero	463:	0.008080
Tablero	464:	0.175864
Tablero	465:	0.136909
Tablero	466:	0.096082
Tablero	467:	0.212344
Tablero	468:	0.128190
Tablero	469:	0.205541
Tablero	470:	0.217116
Tablero	471:	1.252546
Tablero	472:	0.091662
Tablero	473:	0.000000
Tablero	474:	0.212412
Tablero	475:	0.227618
Tablero	476:	0.167532
Tablero	477:	0.120582
Tablero	478:	0.013927

Tablero	479:	0.079214
Tablero	480:	0.293178
Tablero	481:	0.201677
Tablero	482:	0.362758
Tablero	483:	0.006472
Tablero	484:	0.174860
Tablero	485:	0.204008
Tablero	486:	0.113375
Tablero	487:	0.092983
Tablero	488:	0.044883
Tablero	489:	0.042773
Tablero	490:	0.000000
Tablero	491:	0.160392
Tablero	492:	0.123672
Tablero	493:	0.123639
Tablero	494:	1.202558
Tablero	495:	0.185724
Tablero	496:	0.203509
Tablero	497:	0.130505
Tablero	498:	0.016065
Tablero	499:	0.046755
Tablero	500:	0.201280
Tablero	501:	0.343449
Tablero	502:	0.349440
Tablero	503:	0.519085
Tablero	504:	0.147197
Tablero	505:	0.227470
Tablero	506:	0.467819
Tablero	507:	0.157851
Tablero	508:	0.185440
Tablero	509:	0.046790
Tablero	510:	0.097374
Tablero	511:	0.238134
Tablero	512:	0.000000
Tablero	513:	0.004360
Tablero	514:	0.072219
Tablero	515:	0.316543
Tablero	516:	0.151583
Tablero	517:	0.060045
Tablero	518:	0.124480
Tablero	519:	0.304851
Tablero	520:	0.452480
Tablero	521:	0.542256
Tablero	522:	0.000000
Tablero	523:	0.050871
Tablero	524:	0.092078
Tablero	525:	0.156399
Tablero	526:	0.119944
Tablero	527:	0.032517
Tablero	528:	1.369808
Tablero	529:	0.300050
Tablero	530:	0.216480
Tablero	531:	0.125573
Tablero	532:	1.933019
Tablero	533:	0.184463
Tablero	534:	0.121564
Tablero	535:	0.027227
Tablero	536:	0.114564
Tablero	537:	0.066682
Tablero	538:	0.142473
Tablero	539:	0.087617
Tablero	540:	0.239236

Tablero	541:	0.064050
Tablero	542:	0.238055
Tablero	543:	0.261169
Tablero	544:	0.211733
Tablero	545:	0.081549
Tablero	546:	0.050292
Tablero	547:	0.055836
Tablero	548:	0.330905
Tablero	549:	0.370773
Tablero	550:	0.096546
Tablero	551:	0.241514
Tablero	552:	0.102419
Tablero	553:	0.248907
Tablero	554:	0.136562
Tablero	555:	0.197090
Tablero	556:	0.063501
Tablero	557:	0.368395
Tablero	558:	0.186731
Tablero	559:	0.081594
Tablero	560:	0.169009
Tablero	561:	0.157840
Tablero	562:	0.302539
Tablero	563:	0.098112
Tablero	564:	0.294439
Tablero	565:	0.124680
Tablero	566:	0.697199
Tablero	567:	0.098273
Tablero	568:	0.183920
Tablero	569:	0.185181
Tablero	570:	0.069852
Tablero	571:	0.384157
Tablero	572:	0.080169
Tablero	573:	0.034338
Tablero	574:	0.271875
Tablero	575:	0.128441
Tablero	576:	0.000000
Tablero	577:	0.188443
Tablero	578:	0.192022
Tablero	579:	0.135581
Tablero	580:	0.026127
Tablero	581:	2.047186
Tablero	582:	0.215420
Tablero	583:	0.185223
Tablero	584:	0.211677
Tablero	585:	0.153291
Tablero	586:	0.180953
Tablero	587:	0.107535
Tablero	588:	0.187773
Tablero	589:	0.167085
Tablero	590:	0.074670
Tablero	591:	0.325689
Tablero	592:	0.049756
Tablero	593:	0.440541
Tablero	594:	0.072663
Tablero	595:	0.079944
Tablero	596:	0.063604
Tablero	597:	0.081683
Tablero	598:	0.127710
Tablero	599:	0.165908
Tablero	600:	0.107666
Tablero	601:	0.461842
Tablero	602:	0.158594

Tablero	603:	0.517288
Tablero	604:	0.564279
Tablero	605:	0.271448
Tablero	606:	0.145477
Tablero	607:	0.057101
Tablero	608:	0.223896
Tablero	609:	0.161727
Tablero	610:	0.085143
Tablero	611:	0.265490
Tablero	612:	0.420858
Tablero	613:	0.031595
Tablero	614:	0.140662
Tablero	615:	0.556627
Tablero	616:	0.072533
Tablero	617:	0.234197
Tablero	618:	0.125716
Tablero	619:	0.268577
Tablero	620:	0.094551
Tablero	621:	0.227652
Tablero	622:	0.137348
Tablero	623:	0.230825
Tablero	624:	0.040681
Tablero	625:	0.077744
Tablero	626:	0.144709
Tablero	627:	0.078047
Tablero	628:	0.111361
Tablero	629:	0.100585
Tablero	630:	0.031918
Tablero	631:	0.196711
Tablero	632:	0.150824
Tablero	633:	0.403006
Tablero	634:	0.133849
Tablero	635:	0.182631
Tablero	636:	0.637774
Tablero	637:	0.135579
Tablero	638:	0.148696
Tablero	639:	0.072544
Tablero	640:	1.163766
Tablero	641:	0.000000
Tablero	642:	0.191657
Tablero	643:	0.127191
Tablero	644:	0.234452
Tablero	645:	0.140809
Tablero	646:	0.155598
Tablero	647:	0.078822
Tablero	648:	0.164220
Tablero	649:	0.128077
Tablero	650:	0.298030
Tablero	651:	0.088897
Tablero	652:	0.150645
Tablero	653:	0.066913
Tablero	654:	0.426581
Tablero	655:	0.034800
Tablero	656:	0.367748
Tablero	657:	0.044054
Tablero	658:	0.268175
Tablero	659:	0.000000
Tablero	660:	0.097786
Tablero	661:	0.490791
Tablero	662:	0.164948
Tablero	663:	0.115728
Tablero	664:	0.043260

Tablero	665:	0.225803
Tablero	666:	0.223855
Tablero	667:	0.134996
Tablero	668:	0.117158
Tablero	669:	0.329868
Tablero	670:	0.265998
Tablero	671:	0.179546
Tablero	672:	0.025432
Tablero	673:	0.231463
Tablero	674:	0.040186
Tablero	675:	0.138789
Tablero	676:	0.000000
Tablero	677:	0.219993
Tablero	678:	0.049764
Tablero	679:	0.006779
Tablero	680:	0.253953
Tablero	681:	0.166535
Tablero	682:	0.107841
Tablero	683:	0.030442
Tablero	684:	0.048071
Tablero	685:	0.195195
Tablero	686:	0.359386
Tablero	687:	0.276668
Tablero	688:	0.104349
Tablero	689:	0.090852
Tablero	690:	0.477809
Tablero	691:	0.088044
Tablero	692:	0.175053
Tablero	693:	0.208133
Tablero	694:	0.155811
Tablero	695:	0.409583
Tablero	696:	0.168152
Tablero	697:	0.325629
Tablero	698:	0.365111
Tablero	699:	0.210447
Tablero	700:	0.075900
Tablero	701:	0.143101
Tablero	702:	0.149707
Tablero	703:	0.140503
Tablero	704:	0.128678
Tablero	705:	0.351134
Tablero	706:	0.180717
Tablero	707:	0.080731
Tablero	708:	0.612694
Tablero	709:	0.272867
Tablero	710:	0.225406
Tablero	711:	0.312591
Tablero	712:	0.114357
Tablero	713:	0.007760
Tablero	714:	0.083501
Tablero	715:	0.265813
Tablero	716:	0.269712
Tablero	717:	0.235838
Tablero	718:	0.253150
Tablero	719:	0.461412
Tablero	720:	0.070865
Tablero	721:	0.054080
Tablero	722:	0.462311
Tablero	723:	0.186300
Tablero	724:	0.211226
Tablero	725:	0.103168
Tablero	726:	0.142939

Tablero	727:	0.000000
Tablero	728:	0.005623
Tablero	729:	0.047459
Tablero	730:	0.128367
Tablero	731:	0.129930
Tablero	732:	0.186496
Tablero	733:	0.207580
Tablero	734:	0.097149
Tablero	735:	0.159315
Tablero	736:	0.252011
Tablero	737:	1.781402
Tablero	738:	1.312358
Tablero	739:	0.137148
Tablero	740:	0.174128
Tablero	741:	1.202558
Tablero	742:	0.207184
Tablero	743:	0.147139
Tablero	744:	0.175259
Tablero	745:	0.236783
Tablero	746:	0.244194
Tablero	747:	0.116872
Tablero	748:	0.132210
Tablero	749:	0.000000
Tablero	750:	0.054342
Tablero	751:	0.155509
Tablero	752:	0.084577
Tablero	753:	0.195659
Tablero	754:	0.247221
Tablero	755:	0.540453
Tablero	756:	0.151130
Tablero	757:	6.547399
Tablero	758:	0.363864
Tablero	759:	0.089885
Tablero	760:	0.049499
Tablero	761:	0.128183
Tablero	762:	0.077874
Tablero	763:	0.374275
Tablero	764:	0.146411
Tablero	765:	0.073783
Tablero	766:	0.108507
Tablero	767:	0.484456
Tablero	768:	0.080200
Tablero	769:	0.316624
Tablero	770:	0.336573
Tablero	771:	0.172490
Tablero	772:	6.547399
Tablero	773:	0.248997
Tablero	774:	0.109534
Tablero	775:	0.043660
Tablero	776:	0.496281
Tablero	777:	0.069726
Tablero	778:	0.080024
Tablero	779:	0.126134
Tablero	780:	0.183217
Tablero	781:	0.179351
Tablero	782:	0.000000
Tablero	783:	0.850624
Tablero	784:	0.090339
Tablero	785:	0.000000
Tablero	786:	0.132289
Tablero	787:	0.057764
Tablero	788:	0.000000

Tablero	789:	0.143843
Tablero	790:	0.067329
Tablero	791:	0.403006
Tablero	792:	0.015078
Tablero	793:	0.126898
Tablero	794:	0.000000
Tablero	795:	0.164452
Tablero	796:	0.213409
Tablero	797:	0.109674
Tablero	798:	0.104269
Tablero	799:	0.000000
Tablero	800:	0.142464
Tablero	801:	0.000000
Tablero	802:	0.131167
Tablero	803:	0.310922
Tablero	804:	0.174138
Tablero	805:	0.000000
Tablero	806:	0.120195
Tablero	807:	0.148946
Tablero	808:	0.229960
Tablero	809:	0.000000
Tablero	810:	0.465632
Tablero	811:	0.221562
Tablero	812:	0.146943
Tablero	813:	0.091628
Tablero	814:	0.164035
Tablero	815:	0.239415
Tablero	816:	0.060907
Tablero	817:	0.386781
Tablero	818:	0.139664
Tablero	819:	0.599440
Tablero	820:	0.237825
Tablero	821:	0.146162
Tablero	822:	0.070724
Tablero	823:	0.125881
Tablero	824:	0.236800
Tablero	825:	0.166259
Tablero	826:	0.007467
Tablero	827:	0.119087
Tablero	828:	0.157419
Tablero	829:	0.220160
Tablero	830:	0.000000
Tablero	831:	0.159655
Tablero	832:	0.154896
Tablero	833:	0.384410
Tablero	834:	0.124537
Tablero	835:	0.238478
Tablero	836:	0.135541
Tablero	837:	0.448433
Tablero	838:	0.117327
Tablero	839:	0.465632
Tablero	840:	0.129122
Tablero	841:	0.077750
Tablero	842:	0.192300
Tablero	843:	0.000000
Tablero	844:	0.254475
Tablero	845:	0.125058
Tablero	846:	0.143429
Tablero	847:	0.204616
Tablero	848:	0.104875
Tablero	849:	0.440541
Tablero	850:	0.157395

Tablero	851:	0.280656
Tablero	852:	0.827311
Tablero	853:	0.000000
Tablero	854:	0.438687
Tablero	855:	0.109821
Tablero	856:	0.000000
Tablero	857:	0.013115
Tablero	858:	0.184727
Tablero	859:	0.106908
Tablero	860:	0.049477
Tablero	861:	0.134905
Tablero	862:	0.083296
Tablero	863:	0.374451
Tablero	864:	0.209484
Tablero	865:	0.102940
Tablero	866:	0.053657
Tablero	867:	0.147282
Tablero	868:	0.136967
Tablero	869:	0.113732
Tablero	870:	0.155781
Tablero	871:	0.025792
Tablero	872:	0.081636
Tablero	873:	0.046327
Tablero	874:	0.037823
Tablero	875:	0.573069
Tablero	876:	0.076942
Tablero	877:	0.167302
Tablero	878:	0.054815
Tablero	879:	0.130210
Tablero	880:	0.000000
Tablero	881:	0.252351
Tablero	882:	0.271927
Tablero	883:	0.064313
Tablero	884:	0.611195
Tablero	885:	0.119251
Tablero	886:	0.086286
Tablero	887:	0.216077
Tablero	888:	0.000000
Tablero	889:	0.122640
Tablero	890:	0.253444
Tablero	891:	0.075592
Tablero	892:	0.131461
Tablero	893:	0.093457
Tablero	894:	0.215446
Tablero	895:	0.476268
Tablero	896:	0.114223
Tablero	897:	0.499935
Tablero	898:	0.096642
Tablero	899:	0.239287
Tablero	900:	0.181836
Tablero	901:	0.087855
Tablero	902:	0.051239
Tablero	903:	0.113964
Tablero	904:	0.160231
Tablero	905:	0.016112
Tablero	906:	0.124347
Tablero	907:	0.378076
Tablero	908:	0.217870
Tablero	909:	0.060086
Tablero	910:	0.185068
Tablero	911:	0.126846
Tablero	912:	0.172633

Tablero	913:	0.087235
Tablero	914:	0.073075
Tablero	915:	0.107347
Tablero	916:	0.040134
Tablero	917:	0.087413
Tablero	918:	0.000000
Tablero	919:	0.170384
Tablero	920:	0.034952
Tablero	921:	0.309256
Tablero	922:	0.166904
Tablero	923:	0.063219
Tablero	924:	0.377311
Tablero	925:	0.273756
Tablero	926:	0.000000
Tablero	927:	0.162855
Tablero	928:	0.043629
Tablero	929:	0.219619
Tablero	930:	0.106736
Tablero	931:	0.050209
Tablero	932:	0.013201
Tablero	933:	0.079286
Tablero	934:	0.018347
Tablero	935:	0.145106
Tablero	936:	0.099930
Tablero	937:	0.153651
Tablero	938:	0.247562
Tablero	939:	0.402081
Tablero	940:	0.187694
Tablero	941:	0.069218
Tablero	942:	0.056036
Tablero	943:	0.096262
Tablero	944:	0.149137
Tablero	945:	0.056295
Tablero	946:	0.132587
Tablero	947:	0.000000
Tablero	948:	0.125157
Tablero	949:	0.039383
Tablero	950:	0.720276
Tablero	951:	0.226348
Tablero	952:	0.186000
Tablero	953:	0.497285
Tablero	954:	0.080136
Tablero	955:	0.113506
Tablero	956:	0.000960
Tablero	957:	0.107833
Tablero	958:	0.207455
Tablero	959:	0.133656
Tablero	960:	0.172713
Tablero	961:	0.086657
Tablero	962:	0.314504
Tablero	963:	0.077093
Tablero	964:	0.238942
Tablero	965:	0.465632
Tablero	966:	0.239916
Tablero	967:	0.184408
Tablero	968:	0.827311
Tablero	969:	0.149053
Tablero	970:	0.216545
Tablero	971:	0.145785
Tablero	972:	0.184782
Tablero	973:	0.132761
Tablero	974:	0.063881

```

Tablero 975: 0.069023
Tablero 976: 0.095371
Tablero 977: 0.106782
Tablero 978: 0.085126
Tablero 979: 0.088768
Tablero 980: 0.158335
Tablero 981: 0.037020
Tablero 982: 0.297289
Tablero 983: 0.157145
Tablero 984: 0.289299
Tablero 985: 0.135831
Tablero 986: 0.239237
Tablero 987: 0.076280
Tablero 988: 0.137970
Tablero 989: 0.232016
Tablero 990: 0.085672
Tablero 991: 0.103680
Tablero 992: 0.200459
Tablero 993: 0.003245
Tablero 994: 0.336928
Tablero 995: 0.127510
Tablero 996: 0.530010
Tablero 997: 0.330651
Tablero 998: 0.090420
Tablero 999: 0.129561
Tablero 1000: 0.164173

```

```

def plot_bsp_histogram(BSP_HISTOGRAMA, sae_features, bsp_pieces_names, bsp_pieces_gt_tensor,
                       best_features, best_thresholds, best_fls, n_bins=20):

    # Extraer datos de la BSP
    bsp_idx_hist = bsp_pieces_names.index(BSP_HISTOGRAMA)
    neurona_hist = best_features[bsp_idx_hist]
    threshold_hist = best_thresholds[bsp_idx_hist]
    f1_hist = best_fls[bsp_idx_hist]
    f_max_hist = sae_features[:, neurona_hist].max().item()

    # Activaciones y ground truth
    activaciones = sae_features[:, neurona_hist].cpu().numpy()
    gt = bsp_pieces_gt_tensor[:, bsp_idx_hist].cpu().numpy()
    umbral = threshold_hist * f_max_hist

    # Clasificar
    predicho = (activaciones > umbral).astype(int)
    tp_mask = (gt == 1) & (predicho == 1)
    fp_mask = (gt == 0) & (predicho == 1)
    tn_mask = (gt == 0) & (predicho == 0)
    fn_mask = (gt == 1) & (predicho == 0)

    # Bins
    bins = np.linspace(0, activaciones.max(), n_bins + 1)
    bin_centers = (bins[:-1] + bins[1:]) / 2
    bin_width = bins[1] - bins[0]

    tp_counts, _ = np.histogram(activaciones[tp_mask], bins=bins)
    fp_counts, _ = np.histogram(activaciones[fp_mask], bins=bins)
    tn_counts, _ = np.histogram(activaciones[tn_mask], bins=bins)
    fn_counts, _ = np.histogram(activaciones[fn_mask], bins=bins)

    left_mask = bin_centers <= umbral
    right_mask = bin_centers > umbral

```

```

# Graficar
fig, ax = plt.subplots(figsize=(14, 6))

ax.bar(bin_centers[left_mask], tn_counts[left_mask],
       width=bin_width, color='steelblue', alpha=0.9,
       label='TN (GT=0, pred=0)', align='center')
ax.bar(bin_centers[left_mask], fn_counts[left_mask],
       width=bin_width, bottom=tn_counts[left_mask],
       color='lightcoral', alpha=0.9,
       label='FN (GT=1, pred=0)', align='center')
ax.bar(bin_centers[right_mask], tp_counts[right_mask],
       width=bin_width, color='green', alpha=0.9,
       label='TP (GT=1, pred=1)', align='center')
ax.bar(bin_centers[right_mask], fp_counts[right_mask],
       width=bin_width, bottom=tp_counts[right_mask],
       color='orange', alpha=0.9,
       label='FP (GT=0, pred=1)', align='center')

# Números TN
for i, x in enumerate(bin_centers[left_mask]):
    total = tn_counts[left_mask][i] + fn_counts[left_mask][i]
    if tn_counts[left_mask][i] > 0:
        ax.text(x, total + 0.3, str(tn_counts[left_mask][i]),
                ha='center', va='bottom', fontsize=7, color='steelblue')

# Números TP
for i, x in enumerate(bin_centers[right_mask]):
    total = tp_counts[right_mask][i] + fp_counts[right_mask][i]
    if tp_counts[right_mask][i] > 0:
        ax.text(x, total + 0.3, str(tp_counts[right_mask][i]),
                ha='center', va='bottom', fontsize=7, color='green')

# Umbral
ax.axvline(x=umbral, color='red', linewidth=2, linestyle='--',
          label=f'Umbral = {threshold_hist:.1f} × {f_max_hist:.3f} = {umbral:.4f}')

ax.set_xlabel('Activación de la neurona')
ax.set_ylabel('Conteo de tableros')
ax.set_title(f'Histograma BSP: {BSP_HISTOGRAMA} | Neurona: {neurona_hist} | F1: {f1_hist:.4f}')
ax.legend()
ax.text(umbral * 0.5, ax.get_ylim()[1] * 0.95, 'predicho = 0',
       ha='center', fontsize=10, color='gray')
ax.text(umbral + (activaciones.max() - umbral) * 0.5, ax.get_ylim()[1] * 0.95, 'predicho = 1',
       ha='center', fontsize=10, color='gray')

plt.tight_layout()
plt.show()

print(f"\nResumen:")
print(f"  TP: {tp_mask.sum()} | FP: {fp_mask.sum()}")
print(f"  TN: {tn_mask.sum()} | FN: {fn_mask.sum()}")
print(f"  F1: {f1_hist:.4f}")

print(" Función plot_bsp_histogram definida")

```

Función plot_bsp_histogram definida

```

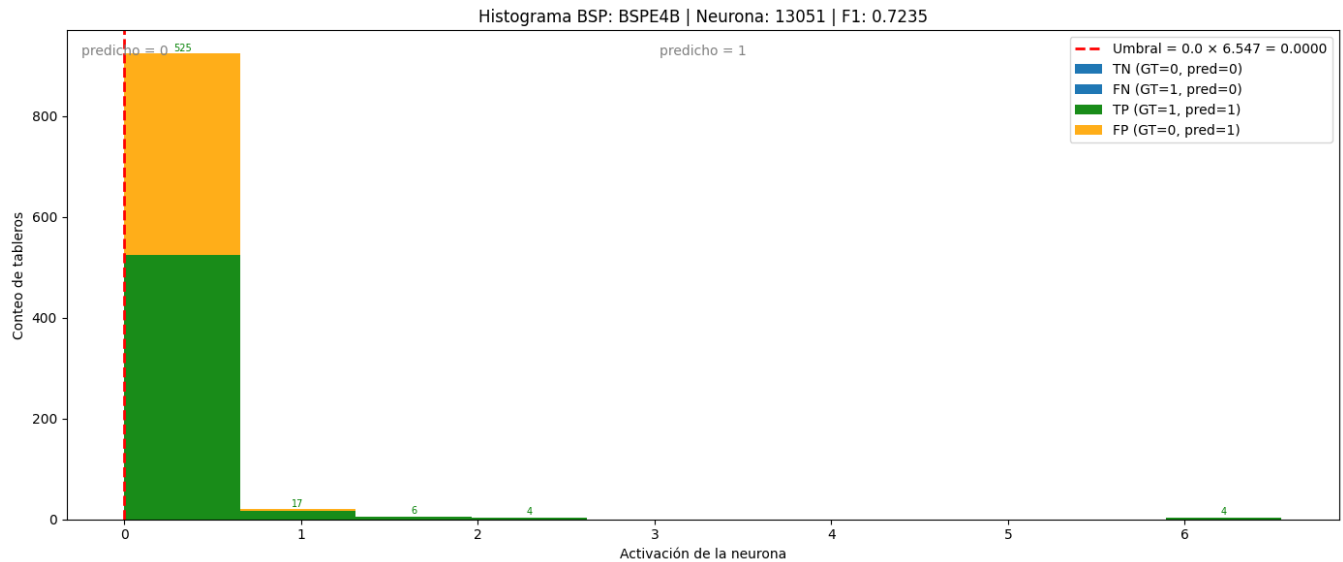
# >>> CAMBIA AQUÍ LA BSP QUE QUIERES VISUALIZAR <
import matplotlib.pyplot as plt
plot_bsp_histogram(
    BSP_HISTOGRAMA      = "BSPE4B",

```

```

sae_features      = sae_features,
bsp_pieces_names  = bsp_pieces_names,
bsp_pieces_gt_tensor = bsp_pieces_gt_tensor,
best_features     = best_features,
best_thresholds   = best_thresholds,
best_f1s         = best_f1s,
n_bins           = 10
)

```



Resumen:

TP: 556 | FP: 402
 TN: 19 | FN: 23
 F1: 0.7235

```

import matplotlib.pyplot as plt
import numpy as np

def plot_f1_distribution(best_f1s, bsp_pieces_names, n_bins=10):
    """
    Gráfico 1: Distribución de F1 scores por BSP.

    Args:
        best_f1s: array numpy con el mejor F1 score de cada BSP
        bsp_pieces_names: lista con los nombres de las BSPs
        n_bins: número de bins del histograma
    """
    fig, ax = plt.subplots(figsize=(10, 6))

    bins = np.linspace(0, 1, n_bins + 1)
    counts, edges = np.histogram(best_f1s, bins=bins)
    bin_centers = (edges[:-1] + edges[1:]) / 2
    bin_width = edges[1] - edges[0]

    bars = ax.bar(
        bin_centers, counts,
        width=bin_width * 0.85,
        color='steelblue',
        edgecolor='white',
        linewidth=0.8
    )

```

```

# Etiquetas encima de cada barra
for bar, count in zip(bars, counts):
    if count > 0:
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.3,
            str(count),
            ha='center', va='bottom', fontsize=9
        )

ax.set_xlabel('F1 Score', fontsize=12)
ax.set_ylabel('Cantidad de BSPs', fontsize=12)
ax.set_title('Distribución de F1 Scores por BSP', fontsize=14)
ax.set_xticks(bins)
ax.set_xticklabels([f'{b:.1f}' for b in bins], rotation=45)
ax.set_xlim(0, 1)

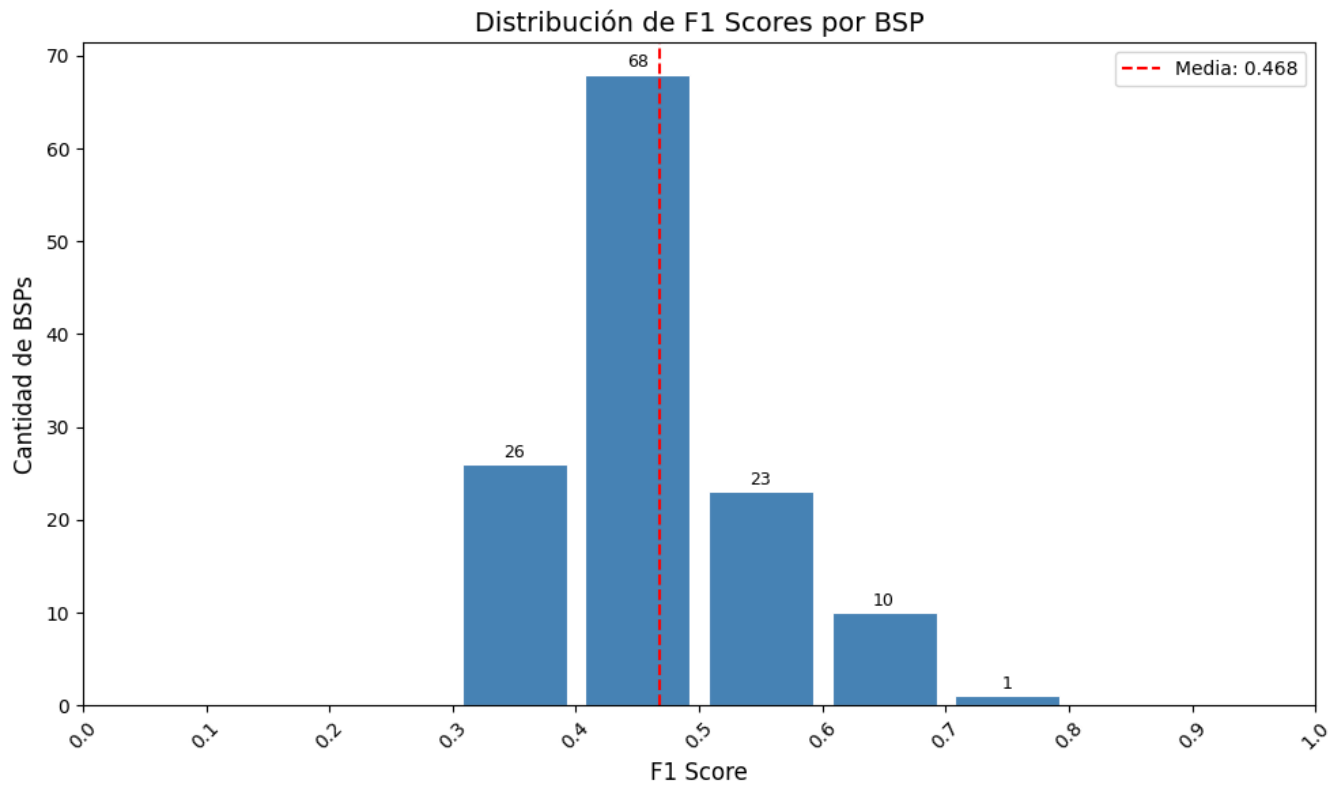
# Línea de la media
mean_f1 = best_f1s.mean()
ax.axvline(mean_f1, color='red', linestyle='--', linewidth=1.5, label=f'Media: {mean_f1:.3f}')
ax.legend(fontsize=10)

plt.tight_layout()
plt.show()

print(f"Total BSPs: {len(best_f1s)}")
print(f"Media F1: {mean_f1:.4f}")
print(f"BSPs con F1 = 0: {(best_f1s == 0).sum()}")
print(f"BSPs con F1 > 0.5: {(best_f1s > 0.5).sum()}")
print(f"BSPs con F1 > 0.9: {(best_f1s > 0.9).sum()}")

# Ejecutar
plot_f1_distribution(best_f1s, bsp_pieces_names)

```



Total BSPs: 128
 Media F1: 0.4678
 BSPs con F1 = 0: 0
 BSPs con F1 > 0.5: 33
 BSPs con F1 > 0.9: 0

```
# Extraer f_max de las features activas
f_max = sae_features.max(dim=0)[0] # (n_features,)
```

```
def plot_max_activation_distribution(best_features, f_max, bsp_pieces_names, n_bins=10):
    """
    Gráfico 2: Distribución de activación máxima de la feature ganadora por BSP.

    Args:
        best_features: array numpy con el índice de la feature ganadora de cada BSP
        f_max: tensor con la activación máxima de cada feature
        bsp_pieces_names: lista con los nombres de las BSPs
        n_bins: número de bins del histograma
    """
    # Obtener f_max de la feature ganadora de cada BSP
    # Ignorar BSPs sin feature ganadora (best_features == -1)
    valid_mask = best_features >= 0
    valid_features = best_features[valid_mask]
    f_max_winners = f_max[valid_features].cpu().numpy()

    fig, ax = plt.subplots(figsize=(10, 6))

    bins = np.linspace(f_max_winners.min(), f_max_winners.max(), n_bins + 1)
    counts, edges = np.histogram(f_max_winners, bins=bins)
    bin_centers = (edges[:-1] + edges[1:]) / 2
    bin_width = edges[1] - edges[0]

    bars = ax.bar(
```

```

    bin_centers, counts,
    width=bin_width * 0.85,
    color='steelblue',
    edgecolor='white',
    linewidth=0.8
)

# Etiquetas encima de cada barra
for bar, count in zip(bars, counts):
    if count > 0:
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.3,
            str(count),
            ha='center', va='bottom', fontsize=9
        )

# Línea de la media
mean_fmax = f_max_winners.mean()
ax.axvline(mean_fmax, color='red', linestyle='--', linewidth=1.5,
            label=f'Media: {mean_fmax:.3f}')
ax.legend(fontsize=10)

ax.set_xlabel('Activación Máxima (f_max)', fontsize=12)
ax.set_ylabel('Cantidad de BSPs', fontsize=12)
ax.set_title('Distribución de Activación Máxima de Features Ganadoras', fontsize=14)
ax.set_xticks(edges)
ax.set_xticklabels([f'{b:.2f}' for b in edges], rotation=45)

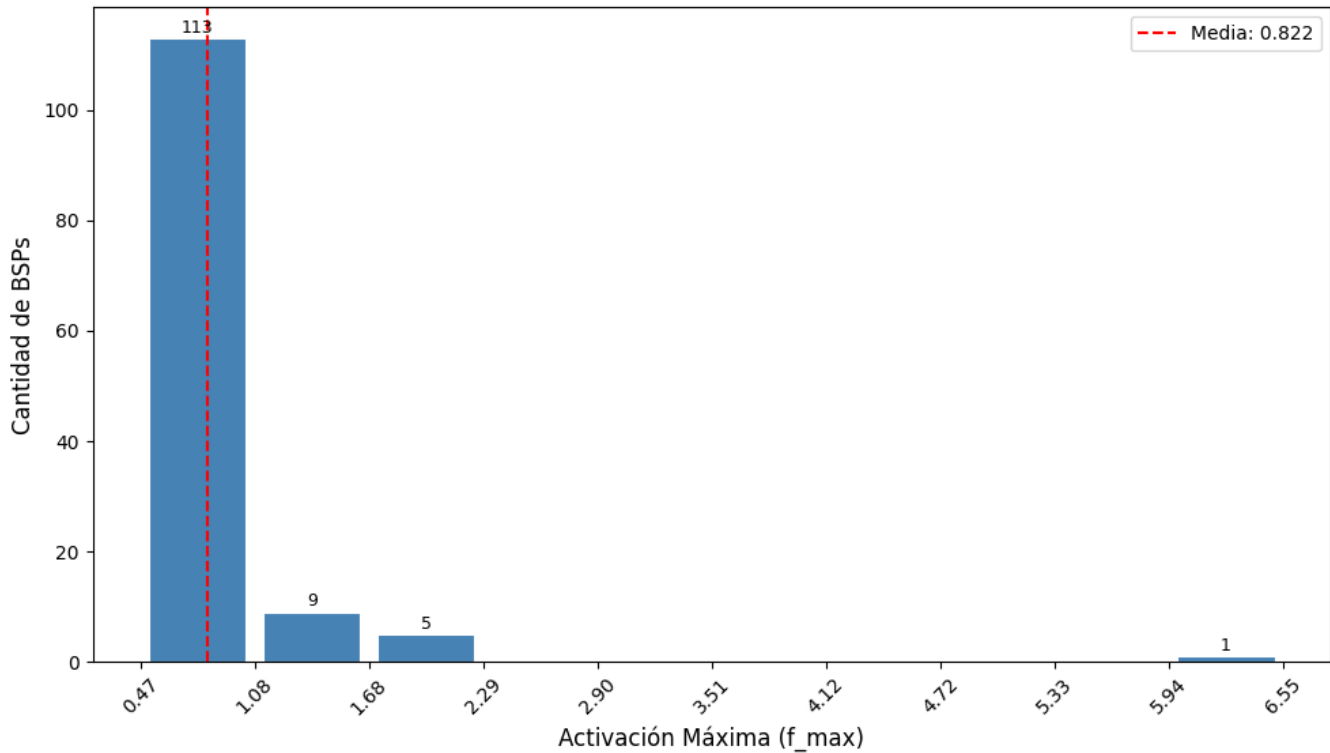
plt.tight_layout()
plt.show()

print(f"Total BSPs con feature ganadora: {valid_mask.sum()}")
print(f"BSPs sin feature ganadora: {(~valid_mask).sum()}")
print(f"Media f_max ganadora: {mean_fmax:.4f}")
print(f"Mínimo f_max: {f_max_winners.min():.4f}")
print(f"Máximo f_max: {f_max_winners.max():.4f}")

# Ejecutar
plot_max_activation_distribution(best_features, f_max, bsp_pieces_names)

```


Distribución de Activación Máxima de Features Ganadoras



Total BSPs con feature ganadora: 128

BSPs sin feature ganadora: 0

Media f_max ganadora: 0.8217

Mínimo f_max: 0.4681

Máximo f_max: 6.5474

```
def plot_threshold_distribution(best_thresholds, bsp_pieces_names):
    """
    Gráfico 3: Distribución de umbrales óptimos por BSP.

    Args:
        best_thresholds: array numpy con el umbral óptimo de cada BSP
        bsp_pieces_names: lista con los nombres de las BSPs
    """
    # Ignorar BSPs sin feature ganadora (best_thresholds == -1)
    valid_mask = best_thresholds >= 0
    valid_thresholds = best_thresholds[valid_mask]

    fig, ax = plt.subplots(figsize=(10, 6))

    # Los umbrales son discretos: 0.0, 0.1, 0.2, ..., 0.9
    threshold_values = np.arange(0.0, 1.0, 0.1)
    counts = [(np.abs(valid_thresholds - t) < 0.05).sum() for t in threshold_values]

    bars = ax.bar(
        threshold_values, counts,
        width=0.07,
        color='steelblue',
        edgecolor='white',
        linewidth=0.8
    )

    # Etiquetas encima de cada barra
```

```

for bar, count in zip(bars, counts):
    if count > 0:
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.3,
            str(count),
            ha='center', va='bottom', fontsize=9
        )

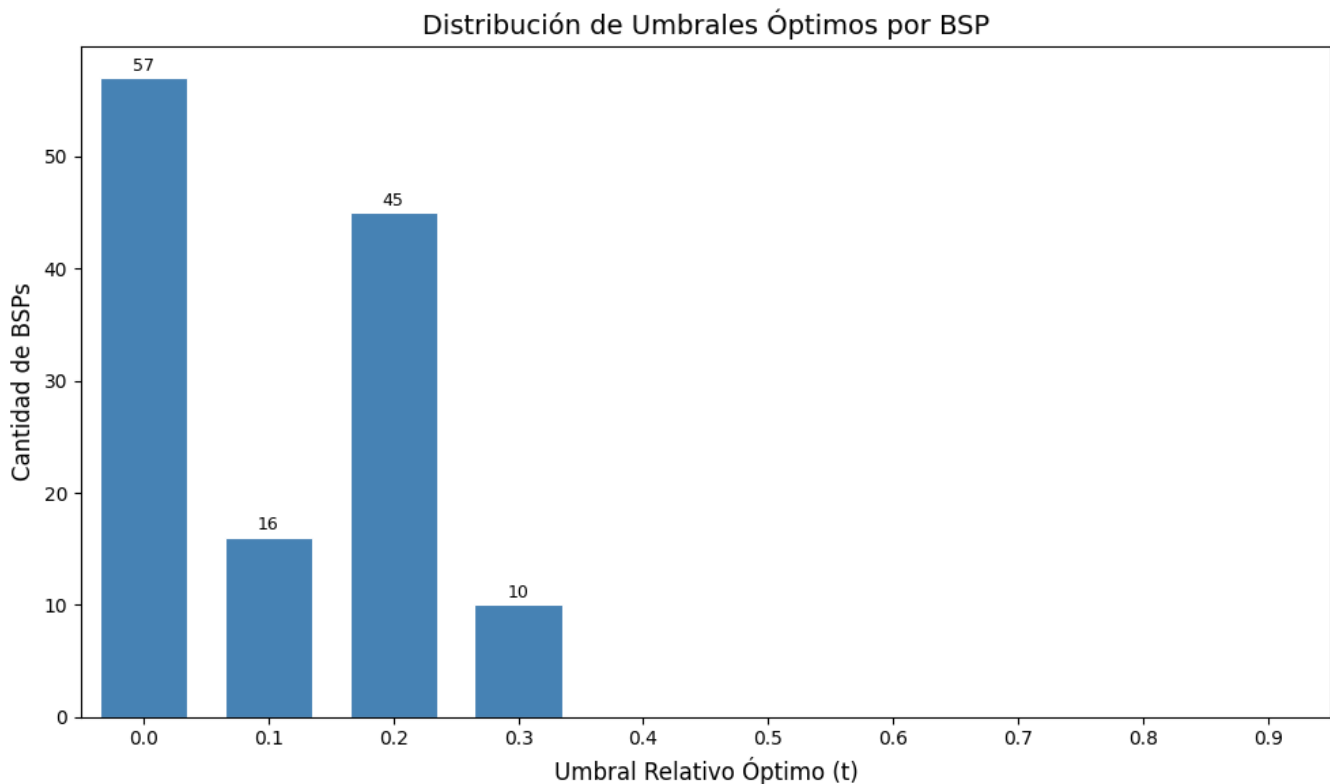
ax.set_xlabel('Umbral Relativo Óptimo (t)', fontsize=12)
ax.set_ylabel('Cantidad de BSPs', fontsize=12)
ax.set_title('Distribución de Umbrales Óptimos por BSP', fontsize=14)
ax.set_xticks(threshold_values)
ax.set_xticklabels([f'{t:.1f}' for t in threshold_values])
ax.set_xlim(-0.05, 0.95)

plt.tight_layout()
plt.show()

print(f"Total BSPs con umbral óptimo: {valid_mask.sum()}")
print(f"BSPs sin feature ganadora: {(~valid_mask).sum()}")
print(f"Umbral más frecuente: {threshold_values[np.argmax(counts)]:.1f}")

# Ejecutar
plot_threshold_distribution(best_thresholds, bsp_pieces_names)

```



Total BSPs con umbral óptimo: 128
 BSPs sin feature ganadora: 0
 Umbral más frecuente: 0.0

0.8 8. Guardar Resultados

```
# Guardar resultados en el directorio de métricas del experimento
results = {
    'coverage': coverage,
    'best_f1s': best_f1s,
    'best_features': best_features,
    'best_thresholds': best_thresholds,
    'bsp_names': bsp_pieces_names,
    'execution_time_seconds': elapsed_time
}

metrics_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)
output_path = metrics_dir / f"coverage_{filter_suffix}.npz"
np.savez(output_path, **results)

print(f" Resultados guardados en:")
print(f"   {output_path}")
```

Resultados guardados en:

c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1-decay_standard\1500games\

0.9 9. Generar PDF con Quarto

```
import subprocess

# Guardar PDF en el mismo directorio que el .npz (metrics/)
pdf_dir = get_metrics_dir(
    NAMING_META['experiment_base_path'],
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    extras_id=extras_id
)

pdf_name = get_report_name(
    NAMING_META['variante'],
    NAMING_META['tecnica'],
    NAMING_META['capa'],
    NAMING_META['juegos'],
    f'coverage_{filter_suffix}',
    extras_id=extras_id
)

notebook_name = 'coverage_gpu_optimized.ipynb'
output_path = pdf_dir / pdf_name

print(f' Generando PDF con Quarto...')
print(f' Archivo de salida: {output_path}')

# Quarto no acepta rutas en --output, solo nombre de archivo
```

```
# Se usa --output-dir para el directorio y --output solo para el nombre
result = subprocess.run(
    f'quarto render {notebook_name} --to pdf --output-dir "{pdf_dir}" --output {pdf_name}',
    shell=True,
    capture_output=True,
    text=True
)

if result.returncode == 0:
    print(f' PDF generado exitosamente: {output_path}')
else:
    print(f' Error al generar PDF:')
    print(result.stderr)
```

Generando PDF con Quarto...

Archivo de salida: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1-decay_standard\1500games\ex1\metrics\sae_standard_l1-decay_l6_1500g_ex1_coverage_absolute.pdf
 PDF generado exitosamente: c:\Users\Esposa\Documents\Repos\othello_world\sae\experiments\layer_06\sae_standard\sae_l1-decay_standard\1500games\ex1\metrics\sae_standard_l1-decay_l6_1500g_ex1_coverage_absolute.pdf